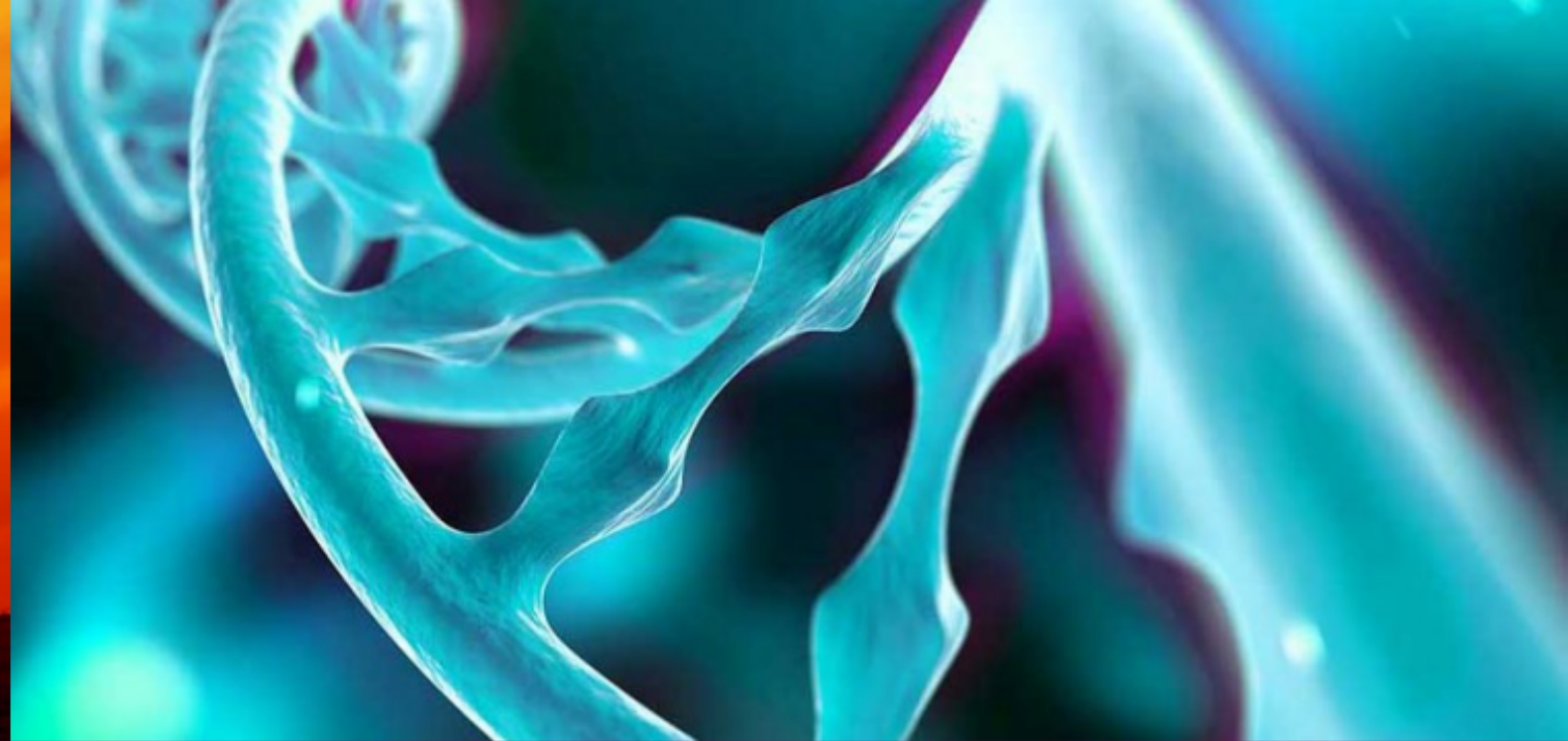
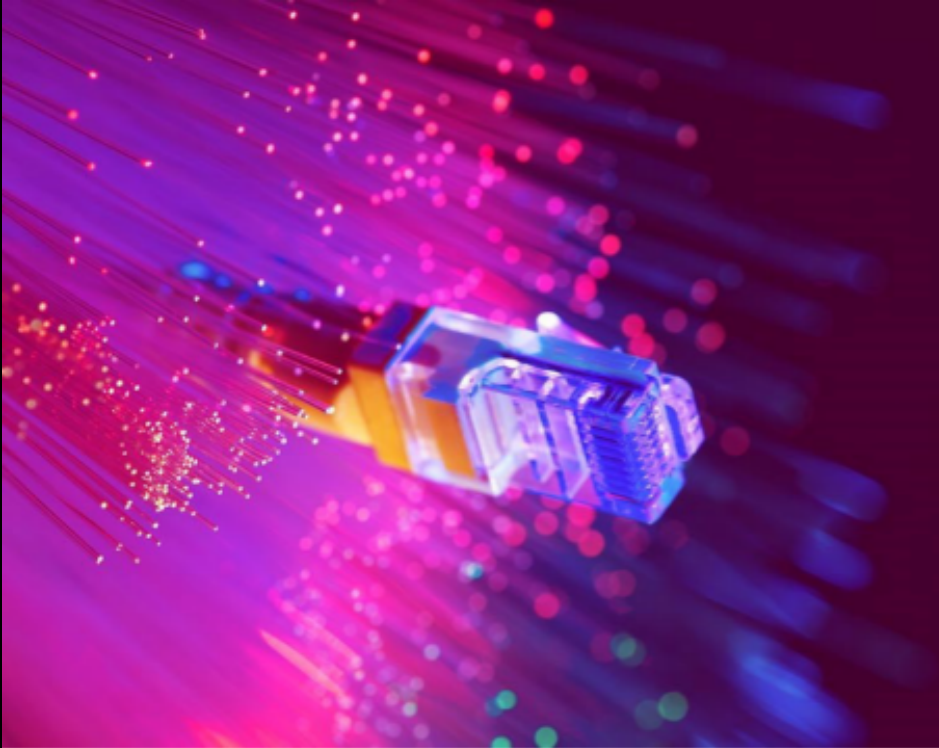




Introduction to GPUs

Student cluster competition

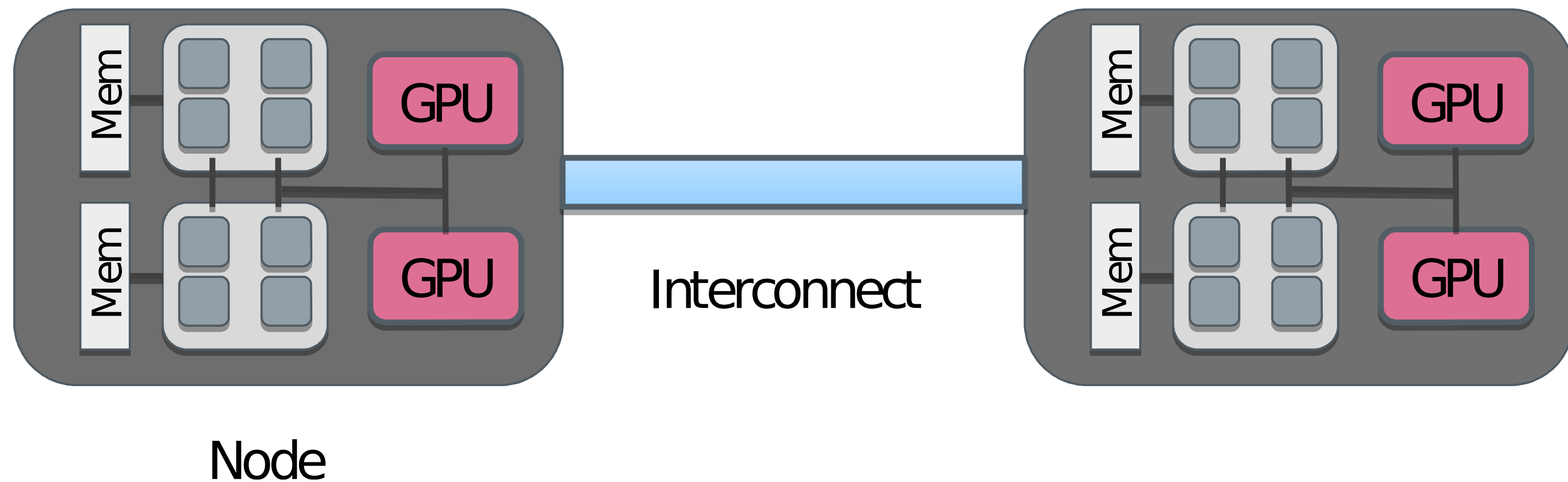


CSC – Finnish expertise in ICT for research, education and public administration

Learning objectives

- Understand the concept of GPUs as co-processors
- Comprehend performance aspects of discrete memory spaces
- Identify amount of parallelism needed with GPUs

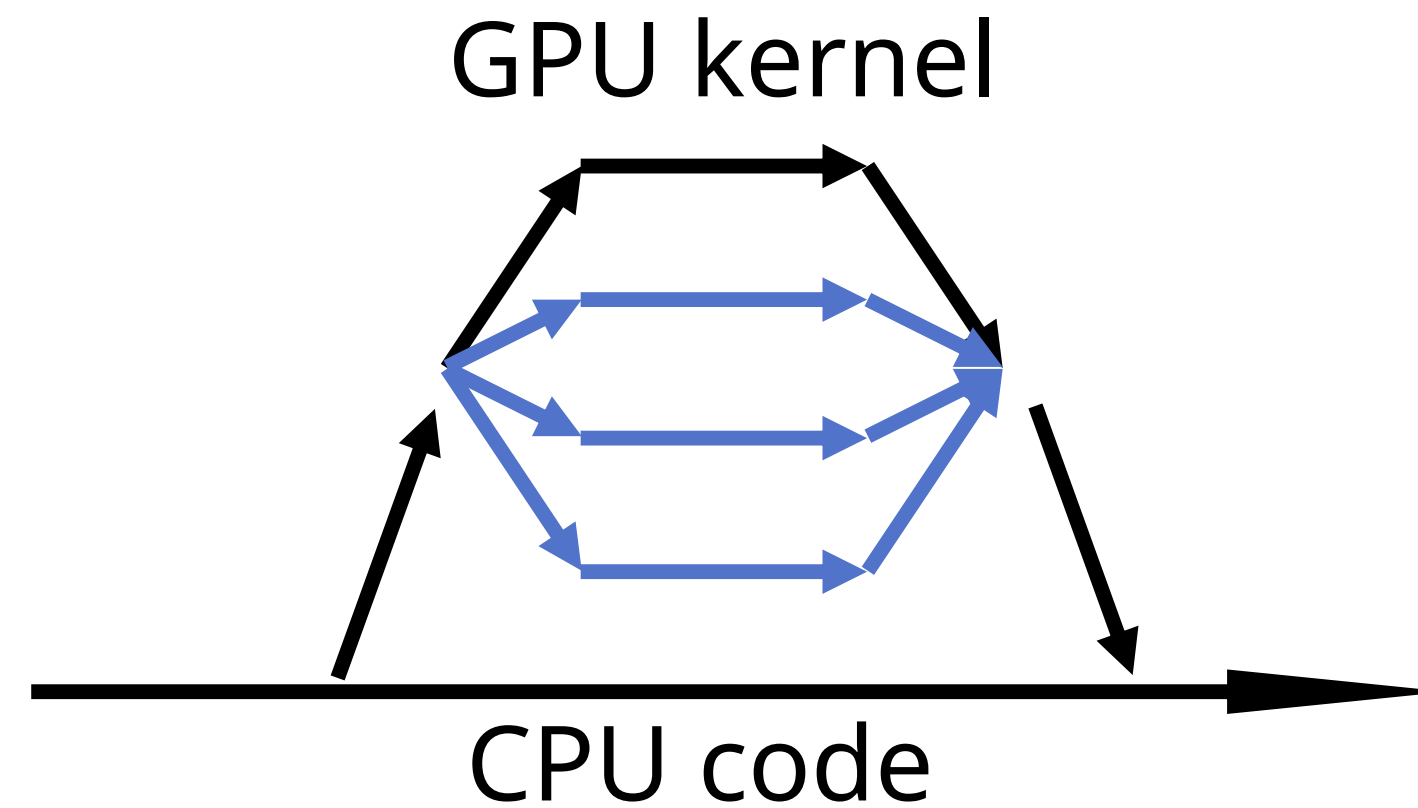
Supercomputer anatomy



GPU operation model

- GPUs are co-processors to the CPU
- CPU controls the work flow:
 - *offloads* computations to GPU by launching *kernels*
 - allocates and deallocates the memory on GPUs
 - handles the data transfers between CPU and GPUs
- GPU kernels run multiple threads
 - Typically much more threads than “GPU cores”
- When using multiple GPUs, CPU runs typically multiple processes (MPI) or multiple threads (OpenMP)

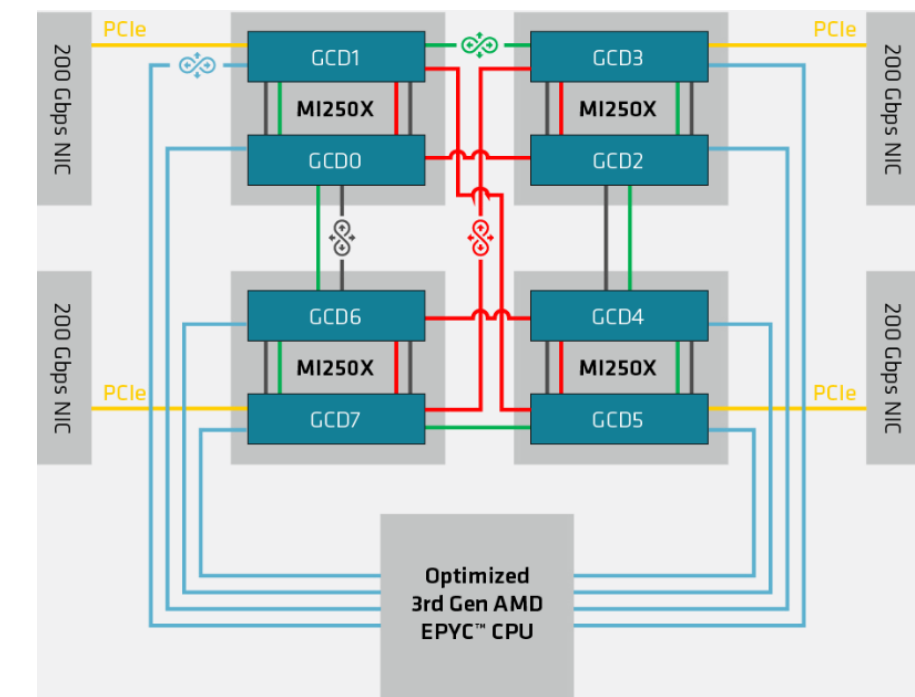
GPU operation model



- CPU launches kernel on GPU
- Kernel execution is normally asynchronous
 - CPU remains active
- Multiple kernels may run concurrently on same GPU

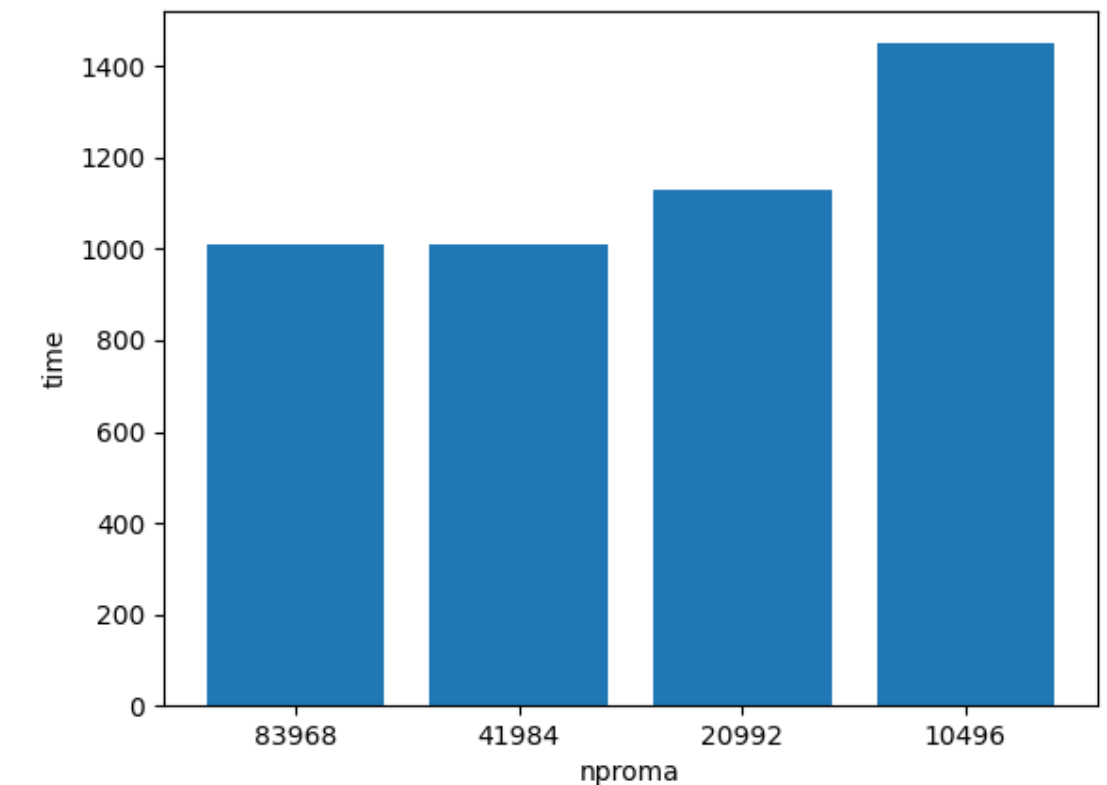
Discrete memory spaces

- On most current systems CPUs and GPUs have discrete memory spaces
 - Data needs to be copied between them via slow bus (in LUMI max 36 GB/s)
- Local memory in GPU is very fast (1.6 TB/s in LUMI)
- Bandwidth between GPUs within a node is 50-200 GB/s
- GPU aware MPI can communicate directly from GPU memories



GPU parallel architecture

- Designed for running tens of thousands of threads simultaneously on thousands of cores
- Very small penalty for switching threads
- Running large amounts of threads hides memory access penalties
- Very expensive to synchronize all threads
- Application needs to expose enough parallelism



GPU programming models

- Directive based models: OpenMP offload, OpenACC)
- Low level C++ (Fortran) extensions: CUDA (CUDA Fortran), HIP
- Performance portability frameworks (C++ only): Kokkos, SYCL
- Machine learning Python frameworks: PyTorch, TensorFlow

Summary

- GPUs are special task co-processor for CPUs
- Data copies between CPU and GPU are slow and may become bottleneck
- Lots of parallelism is needed to fully utilize GPUs